LISTEN: Layered Interpretable Speech-to-Text Evidence Network

An Explainable Question-Answering Framework for Audio and Video Recordings

Tanmoy Saha Reg no.- 23BAI0043
Vaibhav Jain Reg no.- 23BAI0033
Sanjeev Karnatapu Reg no.- 23BAI0003

BCSE419L - Speech and Language Processing Course Project Report
School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu, India

***Abstract*—Long meeting recordings spread their answers over many conversational segments, so questions that need several pieces of evidence are hard to answer with plain semantic retrieval, which also gives little justification for its output. This report presents LISTEN (Layered Interpretable Speech-to-Text Evidence Network), an explainable question-answering framework for audio and video recordings. Faster-Whisper transcribes speech into timestamped segments, spaCy extracts named entities, and MiniLM sentence embeddings are indexed in FAISS for candidate retrieval. The retrieved segments form an evidence graph whose edges capture entity overlap, semantic similarity and temporal proximity, and a two-layer Graph Attention Network (GAT) produces graph-aware relevance scores. The top-ranked segments are passed to Google Gemini, which writes an answer that cites its evidence. For transparency, SHAP gives word-level scores for a retrieval relevance proxy, and counterfactual node removal tests how sensitive an answer is to each evidence segment. The system is built as a FastAPI backend with a React interface. This report describes its design and implementation and sets out a plan for quantitative evaluation.**

***Index Terms*—*Spoken Question Answering, Multi-Hop Reasoning, Graph Attention Network, Explainable AI, Semantic Retrieval, Evidence Graphs, FAISS, SHAP, Counterfactual Explanation*

I. INTRODUCTION

Meetings, interviews, lectures and project discussions are now routinely recorded, and these recordings hold decisions, explanations and dependencies spread over long stretches of conversation. Searching them by hand is slow, especially when the answer is not stated in one place. Spoken question answering (SQA) addresses this by answering natural-language questions from spoken content. Automatic speech recognition (ASR) turns the audio into text so that standard retrieval and language-model techniques can be applied, but finding a relevant passage is not the same as finding a complete answer. Questions about causes, consequences or decisions often need information from several transcript segments.

Consider a meeting in which one speaker says that a supplier failed to deliver components, another says that testing could not begin without them, and a third announces that the product launch is postponed. For the question “Why was the launch delayed?”, a retriever may return only the postponement announcement. The explanation lies in the link between all three statements, so a system needs to represent how pieces of evidence relate to each other.

Dense retrieval ranks passages by semantic similarity to the question, but a similarity score says nothing about whether a passage is sufficient or how it connects to other passages. A graph is a natural way to represent such connections: each node is a timestamped transcript segment, and edges link segments that share named entities, are semantically similar or are close in time. Graph Attention Networks (GATs) [1] learn how much weight each neighbor should receive, and stacking

two layers lets information travel across two hops. The resulting scores are learned importance signals, not proof of causal reasoning.

Users also need to check an answer against the recording. LISTEN therefore pairs each answer with two explanation methods that have different scopes. SHAP [2] gives word-level scores for a similarity-based retrieval relevance proxy; it does not explain how the language model writes its answer. Counterfactual testing removes one evidence segment, reruns the pipeline and compares the new answer with the original; a change shows sensitivity to that segment, not formal causal dependence.

This report presents **LISTEN**, a framework that combines ASR, named entity recognition, dense retrieval with FAISS [3], graph-based evidence ranking, grounded answer generation and post-hoc explanation in one pipeline with a web interface. The main contributions are:

- **Integrated spoken QA pipeline:** from audio or video upload to timestamped, evidence-linked answers.
- **Graph-based evidence reasoning:** an evidence graph over retrieved segments, ranked by a two-layer GAT.
- **Complementary explanations:** SHAP word-level scores for retrieval and counterfactual node removal for answer sensitivity.
- **Evidence-oriented interface:** answers linked to transcript text, timestamps, audio playback and graph scores.

II. LITERATURE REVIEW AND RESEARCH GAP

A. Speech Recognition and Text Representation

ASR converts spoken conversation into text, which is the starting point for conversational question answering. Radford et al. [4] introduced Whisper, a speech recognizer trained with large-scale weak supervision that generalizes across diverse audio domains, and LISTEN runs Whisper models through the Faster-Whisper implementation. Once text is available, passages must be represented so they can be compared with a question. BERT [5] established contextual transformer representations for language understanding, Sentence-BERT [6] adapted them to produce sentence embeddings that can be compared directly, and MiniLM [7] compressed large transformers through self-attention distillation into compact models suited to fast embedding. Transcription and embedding alone, however, do not decide whether a passage gives enough evidence for a question, particularly when the needed information is spread over several statements.

B. Dense Retrieval and Meeting Question Answering

Lexical methods such as BM25 rank passages by term matching, whereas dense retrieval compares embeddings and can match a question to a passage that uses different words. FAISS [3] provides efficient similarity search over such vectors. Similarity is not the same as relevance, though: a passage can discuss the right topic without containing the answer. This matters for meetings, where information is spread across speakers and turns. MeetingQA [8] is an extractive question-answering dataset built on meeting transcripts, and it shows that answers can span several utterances and speakers. Meeting QA therefore needs both candidate retrieval and a way to identify and combine supporting evidence. LISTEN splits these roles into two stages: FAISS retrieves 15 candidates, and a GAT re-ranks them using their relationships.

C. Graph Neural Networks and Multi-Hop Reasoning

Graph neural networks (GNNs) represent data as nodes and edges and update each node from its neighbors. Graph convolutional networks and GraphSAGE aggregate neighborhood information, while GATs [1] add learned attention weights so that different neighbors contribute differently. For multi-hop question answering, sparse-graph approaches using relational graph convolutions have also been explored. The benefit of any graph depends on how it is built: node definition, edge rules and the ranking objective all matter. LISTEN builds its graph only over the segments returned by FAISS and ranks them with a two-layer GAT. Whether this beats retrieval alone has to be shown experimentally against a retrieval-only baseline.

D. Retrieval-Augmented Generation and Evidence Grounding

Retrieval-augmented generation (RAG) [9] gives a language model retrieved context at inference time instead of relying only on what it learned in pretraining. Context does not guarantee a supported answer, however. Graph-based RAG methods use relational structure to improve retrieval and generation. Most relevant here, Tang et al. [10] address long-form audio meeting understanding with a graph-based retrieval-generation agent, which shows that graph methods for meeting audio are already an active direction. LISTEN is therefore not presented as the first such system. Its focus is a specific integration of timestamped transcript retrieval, entity and similarity graphs, GAT ranking, grounded generation and interactive explanation. In its current form Gemini is instructed to cite segment identifiers, and the reliability of those citations must still be measured rather than assumed from the prompt.

E. Explainable AI and Counterfactual Analysis

SHAP [2] attributes a prediction to input features using Shapley values, and LIME explains single predictions through local surrogate models. An explanation can look convincing without reflecting how the model actually decided, a distinction between plausibility and faithfulness. Counterfactual explanations study how changing an input changes the outcome. LISTEN uses SHAP on a retrieval relevance proxy and counterfactual evidence removal on the whole answer pipeline. Each has limits: SHAP here does not explain Gemini's text generation, and a changed answer after removal shows sensitivity but not causal necessity.

F. Summary of Related Work and Research Gap

TABLE I
RELATED WORK AND THE GAP ADDRESSED BY LISTEN

Area	Representative work	Relevance and remaining gap
Speech and embeddings	Whisper [4]; Sentence-BERT [6]; MiniLM [7]	Accurate transcripts and compact embeddings, but no judgment of evidence sufficiency.
Retrieval and meeting QA	FAISS [3]; MeetingQA [8]	Strong candidate retrieval; relationships among retrieved passages are not modeled.
Graph reasoning	GAT [1]	Graphs can help multi-hop QA, but benefit depends on graph design and fair baselines.
Graph RAG and meeting audio	Tang et al. [10]	Graph-based meeting-audio QA already exists; integration with explanation tools varies.
Explainability	SHAP [2]	Explanations differ in scope and need faithfulness checks.

Table 1 summarizes how the literature relates to LISTEN. The cited work gives solid foundations for speech recognition, dense retrieval, graph attention, RAG and explainable AI, and shows that

meeting question answering and graph-based reasoning are active research areas. The opportunity for LISTEN lies in combining these parts in one workflow and evaluating them together. Four issues motivate this. First, dense retrieval does not model relationships among candidate passages, so LISTEN tests whether an evidence graph improves the ranking of supporting segments. Second, generated answers must be traceable, so each answer is linked to timestamped segments whose coverage can be measured. Third, explanation methods differ in scope, so LISTEN combines word-level retrieval explanations with answer-level counterfactual tests. Fourth, a working integration does not prove an improvement, so controlled comparisons with a retrieval-only baseline and ablations of edge types are needed. The central research question is whether graph-aware evidence selection and evidence-level explanations make conversational question answering more reliable and transparent than simpler retrieval-and-generation baselines. This question is to be settled by empirical evaluation, not by the architecture alone.

III. METHODOLOGY

A. System Overview

LISTEN works in three stages, shown in Fig. 1. In Stage A, a recording is converted into timestamped segments that are enriched with entities and embedded into a FAISS index. In Stage B, a question retrieves 15 candidate segments, which are arranged as an evidence graph, ranked by a GAT and passed to Gemini for a grounded answer. In Stage C, the user can request word-level SHAP scores or run a counterfactual test that removes one evidence node and repeats Stage B. Table 2 lists the configuration of the implementation.

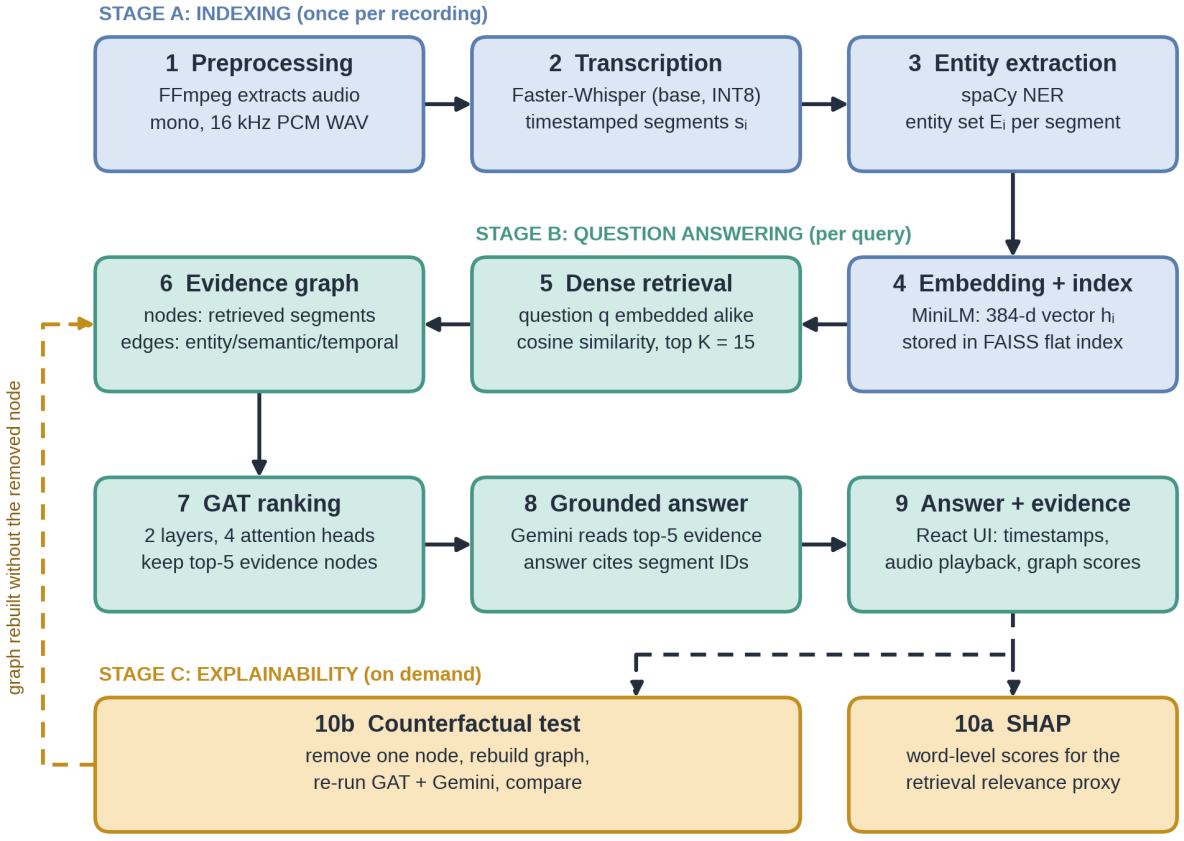


Fig. 1. End-to-end pipeline of LISTEN. Solid arrows show the main flow; dashed arrows show on-demand explanation.

TABLE II
CONFIGURATION OF THE LISTEN IMPLEMENTATION

Component	Configuration
Audio input	FFmpeg converts audio and video (WAV, MP3, FLAC, M4A, OGG, MP4, AVI, MKV, MOV, WEBM) to mono, 16 kHz PCM WAV
Transcription	Faster-Whisper, base model, CPU execution, INT8 computation
Entities	spaCy en_core_web_sm
Embeddings	Sentence Transformers all-MiniLM-L6-v2, 384 dimensions
Retrieval	FAISS flat index (exact search), K = 15 candidates
Evidence graph	At most 25 nodes and 5 edges per node; minimum 1 shared entity; similarity threshold 0.45; semantic edge threshold 0.30; edge weights 0.35 (entity), 0.25 (temporal), 0.40 (semantic)
GAT	PyTorch Geometric; 2 layers, 4 heads; input 384, hidden 128, output 128; dropout 0.2; top 5 evidence segments
Answer generation	Google Gemini API
Application	FastAPI backend with Pydantic schemas; React frontend built with Vite

B. Preprocessing and Transcription

Uploaded audio or video is converted by FFmpeg to mono, 16 kHz PCM WAV. This gives the recognizer a uniform input while keeping the timing needed to map text back to the recording. Faster-Whisper then produces transcript segments with start and end times. Each segment is written as

$$s_i = \left(id_i, t_i^{start}, t_i^{end}, x_i\right) \tag{1}$$

where id_i is the segment identifier, t_i^{start} and t_i^{end} are its time boundaries and x_i is the recognized text. These segments are the basic unit for retrieval, graph construction and explanation.

C. Entity Extraction and Embedding

spaCy [24] with the en_core_web_sm model extracts people, organizations, dates, monetary values and locations from each segment, giving an entity set E_i:

$$E_i = NER\left(x_i\right) \tag{2}$$

The same Sentence-Transformers model f_θ (all-MiniLM-L6-v2) maps every segment and the user question q to a 384-dimensional vector:

$$h_i = f_\theta\left(x_i\right), \quad h_q = f_\theta(q), \quad h_i, h_q \in \mathbb{R}^{384} \tag{3}$$

A shared embedding space lets a question retrieve passages that express the same idea in different words. Entities are used later as hints of a relationship between segments, not as proof that two statements are related.

D. Dense Retrieval with FAISS

Retrieval uses a FAISS flat index, which performs exact nearest-neighbor search over the stored embeddings. The similarity between the question and a segment is the cosine similarity

$$sim\left(q, s_i\right) = \frac{h_q^T h_i}{\|h_q\| \cdot \|h_i\|} \tag{4}$$

and the candidate set contains the K = 15 most similar segments:

$$C_q = TopK_{s_i \in S} sim\left(q, s_i\right), \quad K = 15 \tag{5}$$

where S is the set of indexed segments. Retrieval is only a candidate filter: a high similarity does not mean a segment is enough to answer the question, which is why a separate ranking stage follows.

E. Evidence Graph Construction

The retrieved segments become the evidence graph $G_q = (V_q, E_q)$, with one node per segment. The node feature is the segment's MiniLM embedding, $x_i = h_i$, and each node keeps its identifier, text, timestamps and entities. At most 25 segments enter a graph. Three kinds of edges are considered:

- **Entity edges:** two segments share at least one named entity.
- **Semantic edges:** the embedding similarity of two segments passes the configured thresholds (0.45 and 0.30).
- **Temporal edges:** the two segments are close in conversational time or context.

Each node keeps at most five edges, and the three edge types carry weights of 0.35 (entity), 0.25 (temporal) and 0.40 (semantic). These weights are configuration choices and have not been tuned empirically. Shared entities or similar wording do not guarantee that two segments are really related, so the graph encodes possible relationships only.

F. Graph Attention Network for Evidence Ranking

The GAT, implemented in PyTorch Geometric, takes the evidence graph and returns node and edge relevance scores. It has two layers with four attention heads (input 384, hidden and output 128, dropout 0.2). For node i at layer l, an attention function $a^{(l)}$ scores each neighbor j:

$$e_{ij}^{(l)} = a^{(l)}\left(h_i^{(l)}, h_j^{(l)}\right) \tag{6}$$

The scores are normalized over the neighborhood $N(i)$:

$$\alpha_{ij}^{(l)} = \frac{exp\left(e_{ij}^{(l)}\right)}{\sum_{k \in N(i)} exp\left(e_{ik}^{(l)}\right)} \tag{7}$$

and the node representation is updated as a weighted sum of transformed neighbor representations:

$$h_i^{(l+1)} = \sigma\left(\sum_{j \in N(i)} \alpha_{ij}^{(l)} W^{(l)} h_j^{(l)}\right) \tag{8}$$

where $W^{(l)}$ is a learnable matrix and σ a nonlinear activation. These equations give the general GAT mechanism [1]; the exact operations follow the layer configuration of the implementation. One layer gathers information from direct neighbors and a second layer adds nodes two edges away. In the earlier example, the segments about the supplier delay and the launch postponement can influence each other through the testing segment even if they are not directly linked. This supports multi-hop-aware ranking but does not guarantee that a correct causal chain is found. The top K = 5 nodes form the evidence set:

$$R_q = TopK_{v_i \in V_q} score_{GAT}\left(v_i\right), \quad K = 5 \tag{9}$$

G. Grounded Answer Generation

The text, timestamps, ranks and GAT scores of the selected segments are assembled into an evidence context. Google Gemini receives the question and this context, with instructions to answer from the evidence and to cite segment numbers:

$$\hat{y} = Gemini\left(q, R_q\right) \tag{10}$$

Keeping the stages separate keeps their roles clear: FAISS finds candidates, the GAT ranks them and Gemini only words the answer. The grounding instruction lowers the chance of unsupported statements but does not guarantee a correct answer.

H. Explainability and Evidence Validation

1) SHAP-based retrieval explanation:

SHAP [2] attributes a model output to input features using Shapley values. In LISTEN the output is a similarity-based retrieval relevance function *f*, and the features *w* are the words of a retrieved segment. The contribution of word *w* is

$$\varphi_w = \sum_{S \subseteq F \setminus \{w\}} \frac{|S|!(|F|-|S|-1)!}{|F|!} [f(S \cup \{w\}) - f(S)] \tag{11}$$

where *F* is the feature set and *S* a subset of it; the exact explainer and masking strategy determine how this is approximated in practice. The scores appear as highlighted words in the interface. They describe retrieval relevance only, not the reasoning of the GAT or the text generation of Gemini.

2) Counterfactual evidence-removal test:

To test whether an answer depends on a particular segment, the user selects an evidence node v_i. The modified graph is

$$G_q^{'} = G_q \setminus \{v_i\} \tag{12}$$

and the backend carries out the following steps:

1. Receive the meeting ID, question, original answer and selected node, and retrieve the FAISS candidates again.
2. Check that the node belongs to the evidence set and remove it from the candidates.
3. Rebuild the graph, rerun GAT inference and generate a new answer.
4. Recompute SHAP scores for the remaining evidence.
5. Compare the original answer $\hat{y}$ with the new answer $\hat{y}_{-i}$ and return an answer-change indicator:

$$\Delta y = Compare\left(\hat{y}, \hat{y}_{-i}\right) \tag{13}$$

A changed answer suggests that the removed segment mattered, while an unchanged answer suggests that other evidence was sufficient. Neither outcome proves that a segment is causally necessary or irrelevant, because redundancy among segments, graph reconstruction and generation variability also affect the result.

I. Implementation and User Interaction

The backend is written in Python with FastAPI and exposes endpoints for meeting upload, question answering, counterfactual validation and audio-clip retrieval, with Pydantic schemas for requests and responses. The React (Vite) frontend lets the user upload recordings, ask questions, read answers

with evidence references, inspect graph nodes and GAT scores, view SHAP-highlighted text, play the audio of a selected segment and run counterfactual tests.

IV. RESULTS AND DISCUSSION

The web application is deployed and publicly accessible at listen.vaibhavjain.click.

A. System Demonstration

The implemented system runs end to end, from recording upload to an evidence-linked answer and its explanations. A recorded demonstration and screenshots of the main screens are given below.

Demo video:
https://drive.google.com/file/d/1r-Vepi54ZoQkaH3Erb4P5y1y0idMkryk/view?usp=drive_link

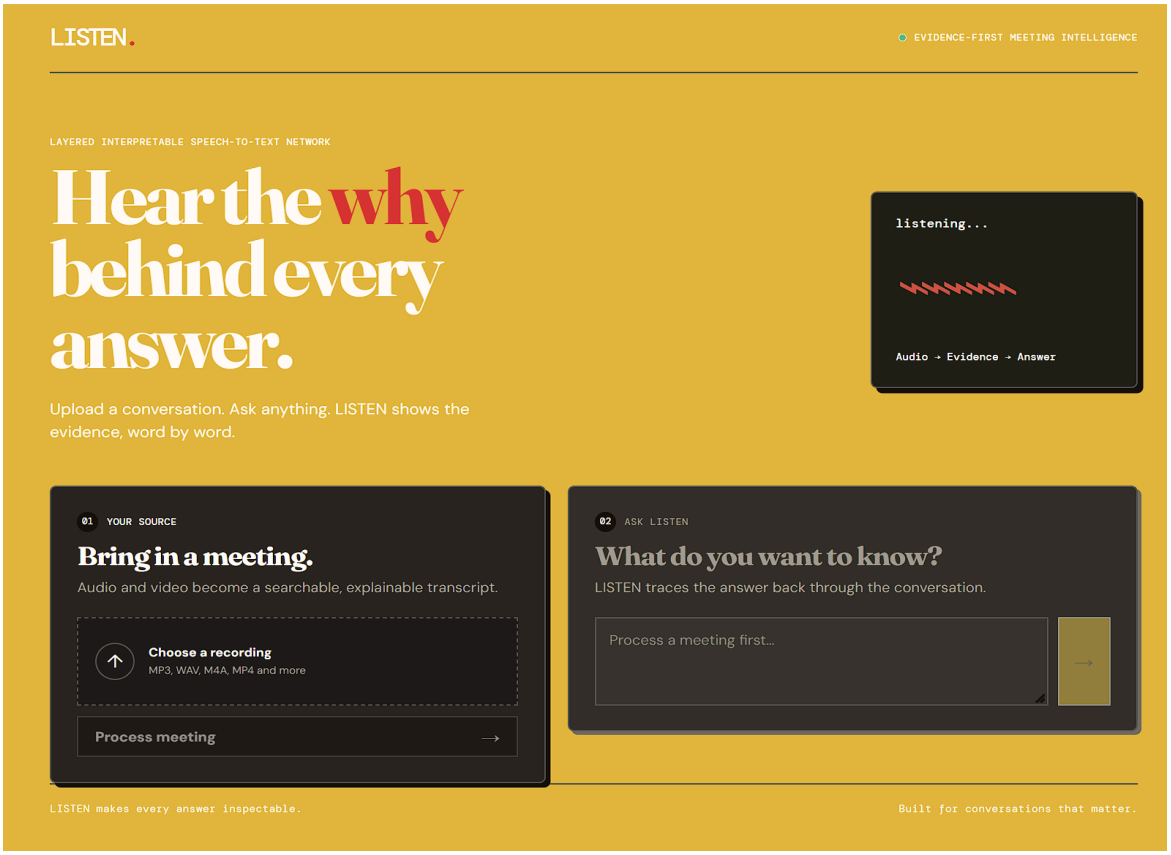


Fig. 2(a). Upload screen and processed transcript segments

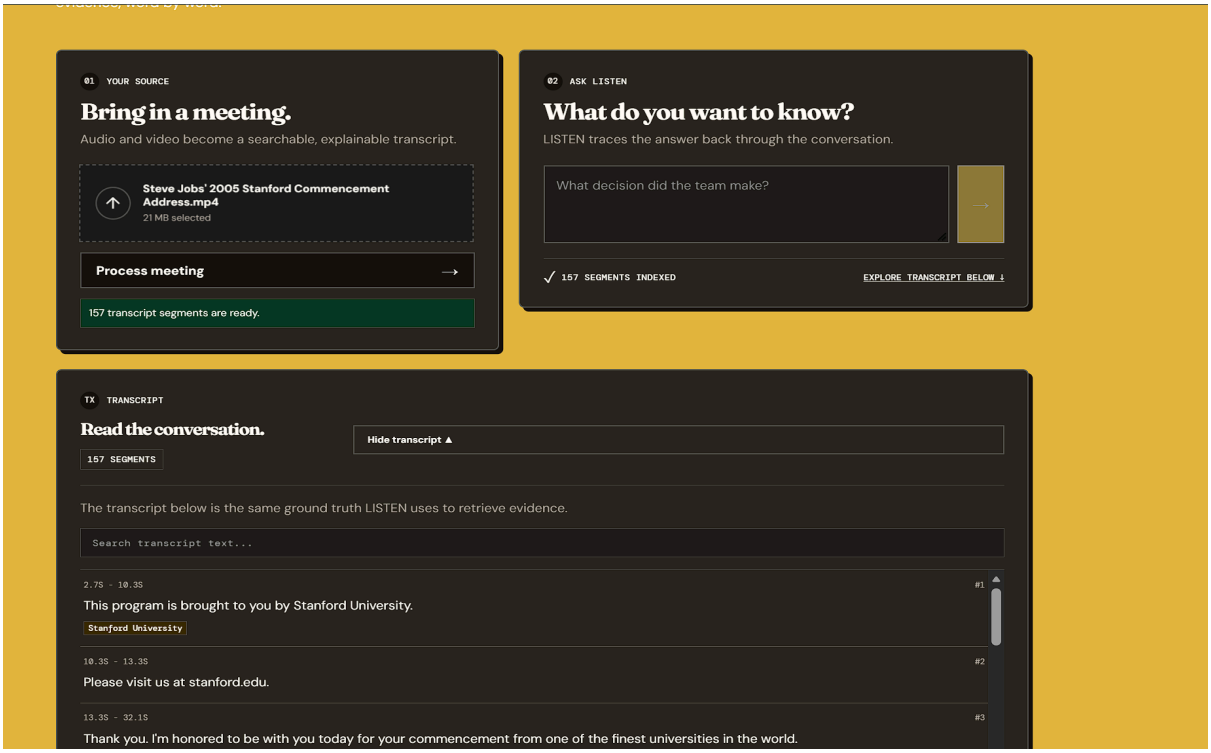


Fig. 2(b). Video processed and generated transcripts

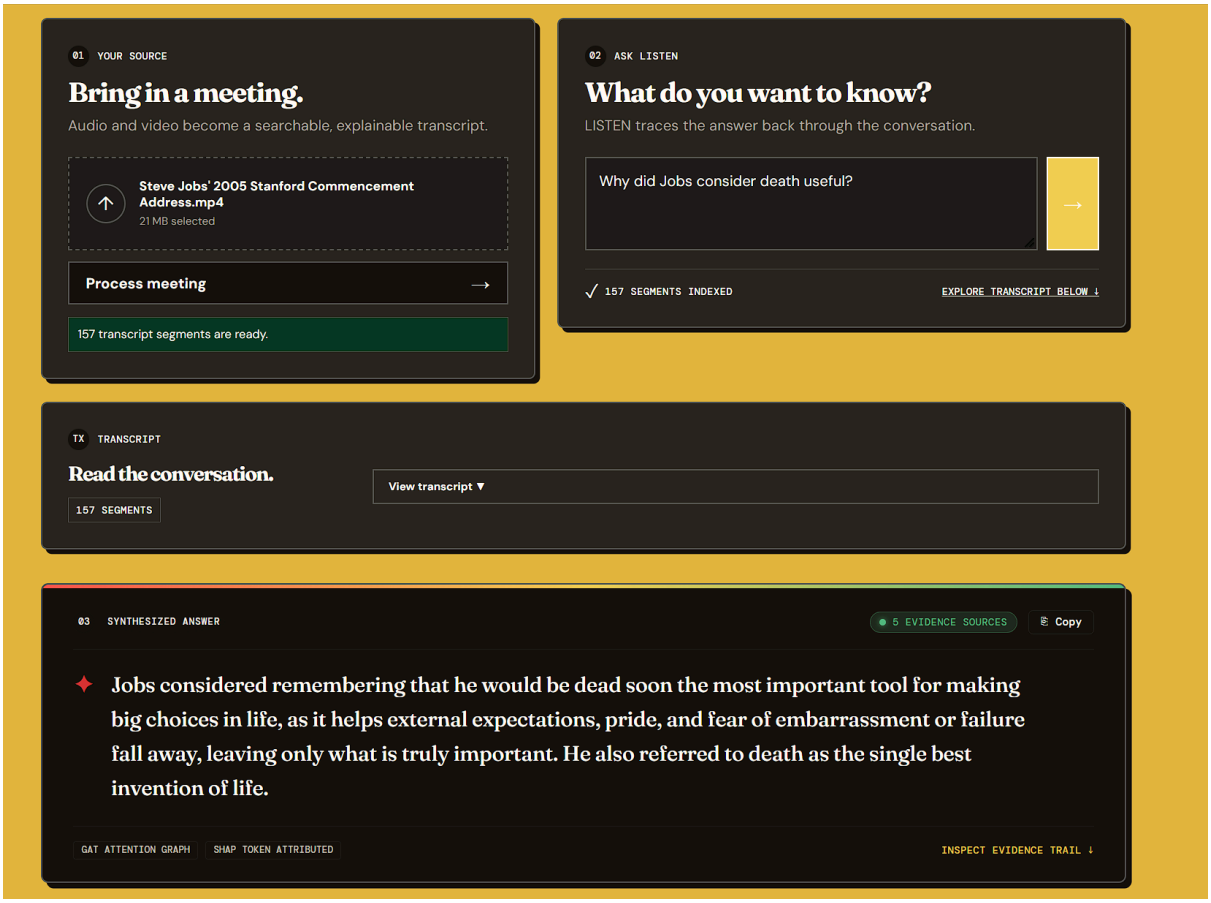


Fig. 2(c). Question, generated answer and cited evidence

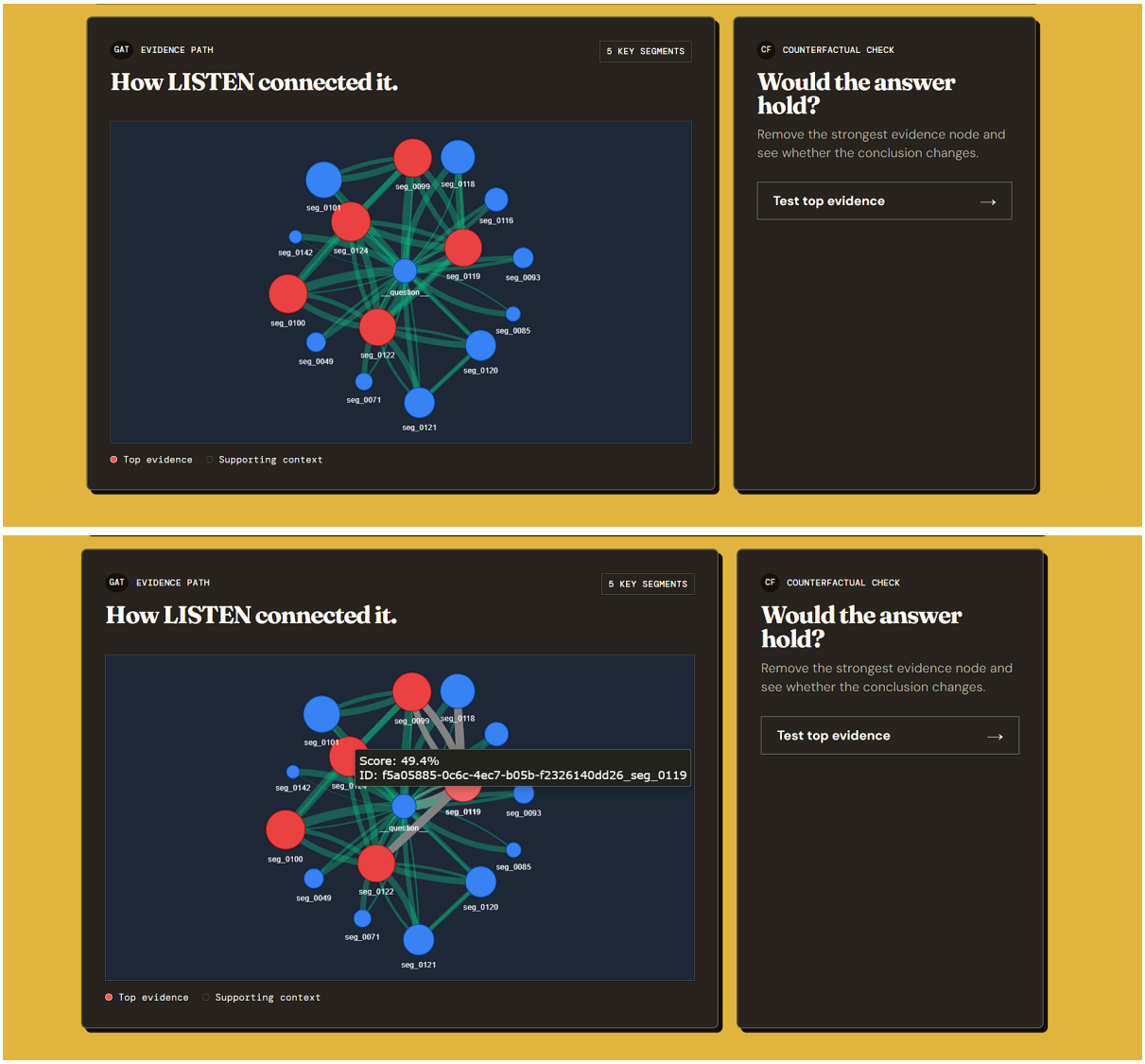


Fig. 2(d). Evidence graph with GAT node score

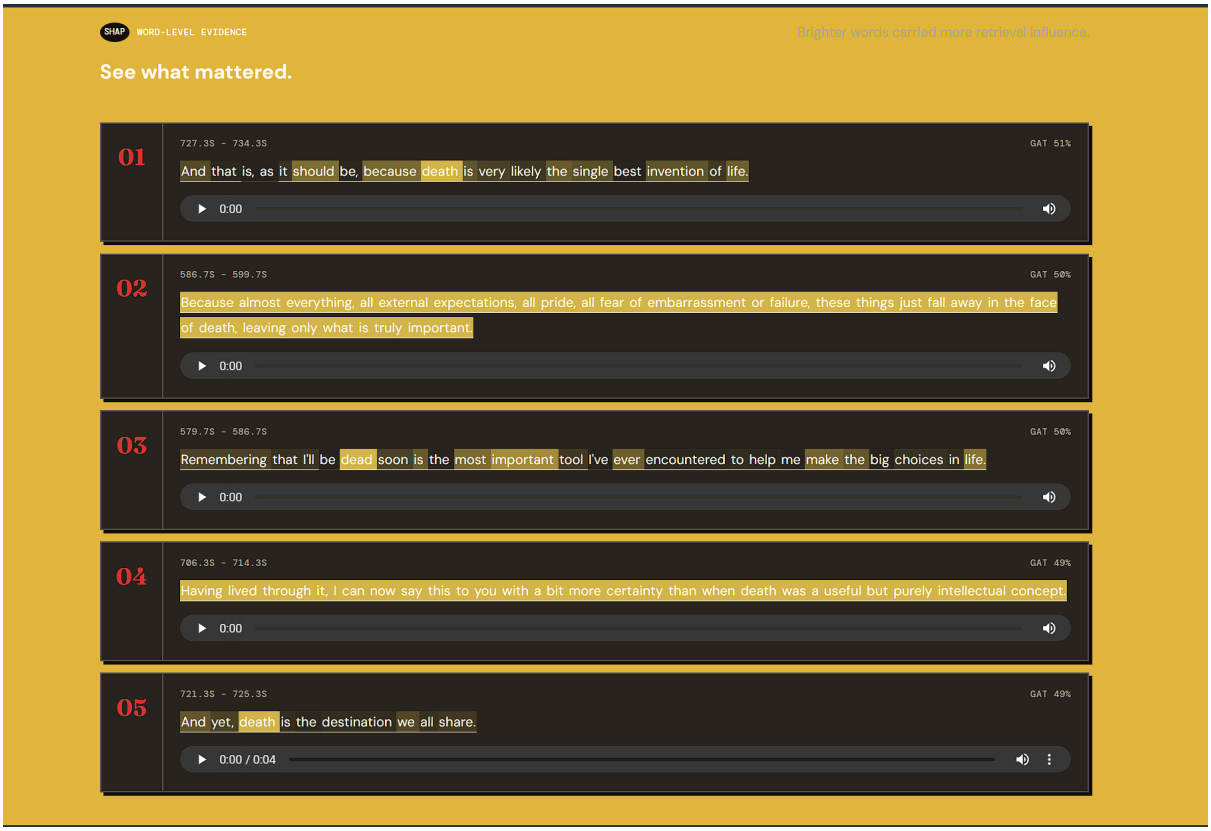


Fig. 2(e). SHAP-highlighted words

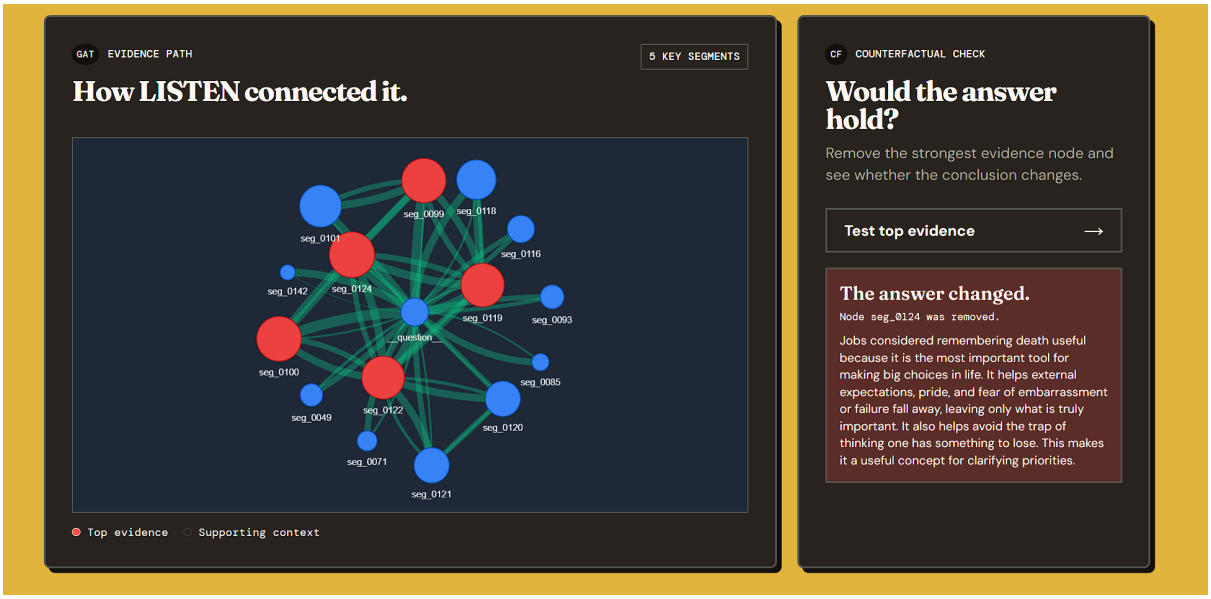


Fig. 2(f).Counterfactual comparison

B. Implementation Outcomes

The working application provides the following capabilities:

- Upload of audio and video recordings, transcription into timestamped segments and display of the number of processed segments.
- Natural-language questions answered with Gemini from the five segments ranked highest by the GAT, with segment references.

- Inspection of graph nodes and GAT scores, and playback of the audio interval for any selected segment.
- SHAP-highlighted transcript words and a counterfactual node-removal test with an answer-change indicator.

Quantitative benchmark results are not included in this version of the report. The next section therefore states how the components are to be evaluated, and the conclusions drawn here concern the design and the working implementation, not measured accuracy gains.

C. Evaluation Plan

Table 3 lists the metrics proposed for each component. Retrieval and evidence ranking can be scored against annotated supporting segments, answer quality against reference answers using Exact Match and token-level F1, and explanations through feature-ablation and counterfactual tests.

TABLE III
PROPOSED EVALUATION OF LISTEN

Aspect	Metrics	Comparison
Retrieval	Recall@K, Precision@K, MRR; share of questions with all supporting segments retrieved	FAISS candidates against annotated evidence
Evidence ranking	Evidence recall and precision at 5; multi-hop supporting-fact coverage	FAISS only against FAISS + GAT
Answer quality	Exact Match, token-level F1, citation accuracy, share of unsupported claims	FAISS + Gemini against full LISTEN
SHAP	Change in relevance score when top-attributed words are removed	Top-attributed against random words
Counterfactual	Answer-change rate and answer similarity after removal	Supporting against non-supporting evidence

The baselines are FAISS + Gemini (dense retrieval only), FAISS + GAT + Gemini (adds graph ranking) and the full system. An ablation can additionally remove each edge type (entity, semantic, temporal) and vary the number of retrieved candidates and selected segments. All configurations should use the same questions, reference answers and data splits, and retrieval, answer quality and explanation faithfulness should be measured separately so that gains in one are not assumed for the others. The GAT training data, loss and optimizer must be documented alongside these results.

D. Discussion and Limitations

The main advantage of the design is the separation of candidate retrieval, graph ranking, answer generation and explanation into distinct stages. Errors can then be traced to transcription, retrieval, graph construction or generation, and each stage can be evaluated on its own. The interface keeps every answer tied to timestamped evidence that the user can read and replay.

Several factors limit what the system can promise. Transcription errors propagate into entities and embeddings. Retrieval may miss needed passages, which limits what the graph can reason over. Edges may link segments that merely share a name or similar wording. GAT scores do not prove correct reasoning, and generated answers may still contain unsupported statements. SHAP covers only the retrieval proxy, and counterfactual results can vary because the whole pipeline is rerun after removal.

V. CONCLUSION

This report presented LISTEN, an explainable question-answering framework for audio and video recordings of conversations. Faster-Whisper turns recordings into timestamped segments, MiniLM embeddings and FAISS retrieve candidate passages, and an evidence graph with a two-layer GAT ranks the candidates by their relationships. Google Gemini then writes an answer from the selected evidence. SHAP word-level scores and counterfactual node removal let users inspect which words and segments the retrieval and the answer depend on, and the web interface links every answer to transcript text, timestamps and audio.

The framework has clear limits. Semantic similarity does not guarantee relevance, graph attention does not establish causality, SHAP is restricted to the retrieval proxy, and counterfactual changes show sensitivity rather than causal dependence. Its benefit over retrieval alone has not yet been measured and has to be established through the evaluation described above. LISTEN is offered as an integrated, inspectable pipeline for evidence-oriented conversational question answering.

VI. FUTURE WORK

1. **Quantitative evaluation:** benchmark on annotated conversational datasets with questions, reference answers and supporting segments, using Recall@K, MRR, answer F1 and evidence coverage, and document the GAT training procedure.
2. **Hybrid retrieval:** combine dense embeddings with BM25 to handle exact names, dates and numbers that similarity search can miss.
3. **Stronger multi-hop reasoning:** explicit evidence-path extraction and relation-aware graph models that separate contextual links from causal ones.
4. **Explanation faithfulness:** feature-ablation tests for SHAP and controlled counterfactual tests with relevant and irrelevant passages.
5. **Speech processing:** speaker diarization, multilingual transcription and better handling of noisy or overlapping speech.
6. **Robustness and deployment:** evidence-sufficiency checks, abstention when context is inadequate, scalable vector stores, locally hosted language models, access control and privacy-preserving storage.

REFERENCES

[1] P. Veličković et al., "Graph Attention Networks," in International Conference on Learning Representations (ICLR), 2018. [Online]. Available: https://arxiv.org/abs/1710.10903

[2] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in Advances in Neural Information Processing Systems, vol. 30, pp. 4765-4774, 2017. [Online]. Available: https://arxiv.org/abs/1705.07874

[3] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," IEEE Transactions on Big Data, vol. 7, no. 3, pp. 535-547, 2021. doi: 10.1109/TBDATA.2019.2921572

[4] A. Radford et al., "Robust Speech Recognition via Large-Scale Weak Supervision," in Proceedings of the 40th International Conference on Machine Learning (ICML), vol. 202, pp. 28492-28518, 2023. [Online]. Available: https://proceedings.mlr.press/v202/radford23a.html

[5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proceedings of NAACL-HLT, pp. 4171-4186, 2019. doi: 10.18653/v1/N19-1423

[6] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proceedings of EMNLP-IJCNLP, pp. 3982-3992, 2019. doi: 10.18653/v1/D19-1410

[7] W. Wang et al., "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers," in Advances in Neural Information Processing Systems, vol. 33, 2020. [Online]. Available: https://arxiv.org/abs/2002.10957

[8] A. Prasad, T. Bui, S. Yoon, H. Deilamsalehy, F. Dernoncourt, and M. Bansal, "MeetingQA: Extractive Question-Answering on Meeting Transcripts," in Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 15000-15025, 2023. doi: 10.18653/v1/2023.acl-long.837

[9] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems, vol. 33, pp. 9459-9474, 2020. [Online]. Available: https://arxiv.org/abs/2005.11401

[10] Q. Tang, D. Zhang, S. Li, and G. Zhou, "Don't Just Listen, Try Planning: Graph-based Retrieval-Generation Agent for Long-form Audio Meeting Understanding," in Findings of the Association for Computational Linguistics: ACL 2026, pp. 20715-20742, 2026. doi: 10.18653/v1/2026.findings-acl.1038